Here are the **complete, well-structured, and accurate answers** for the class test based on the *SriBank PLC* scenario and the architectural patterns notes you provided:

---

1. Architecture for the Internet and Mobile Banking Platform

a. Layers and Tiers

- **Presentation Tier (Client Layer)**
    - Mobile Apps (iOS/Android)
    - Web App (Responsive web app)
- **Business Logic Tier (Middle Layer)**
    - Application Server (Handles all business rules and logic)
    - Security (Authentication, Authorization)
- **Data Tier (Back-End Layer)**
    - Relational Database (Customer Data, Transactions)
    - External Systems (Core Banking, Credit Card, etc.)

---

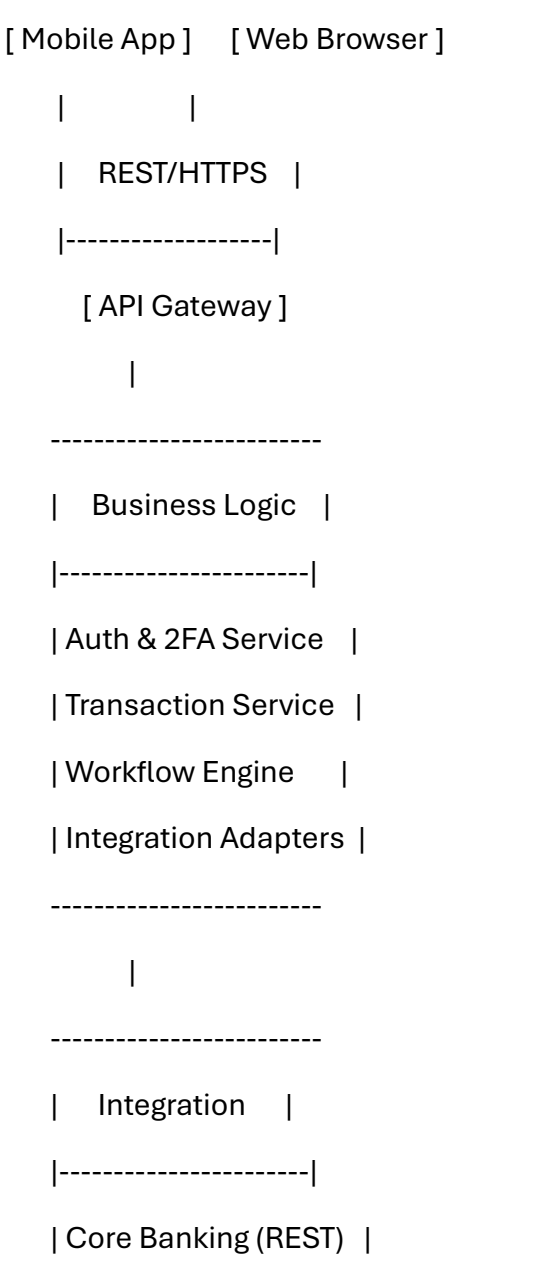
b. Components of Each Layer

- **Presentation Tier**
    - Web Browser (React/Vue front-end)
    - Mobile App (React Native/Flutter)
    - Controllers for input validation
- **Business Logic Tier**
    - REST API Server (Spring Boot/.NET)
    - Authentication Service (2FA, OAuth2)
    - Workflow Engine (for account creation, loan requests)
    - Service Integrators (REST, SOAP, ISO 8583 adapters)
- **Data Tier**

- RDBMS (PostgreSQL/MySQL)
- Core Banking System (REST)
- Inter-bank Gateway (ISO 8583)
- Credit Card System (SOAP)
- Internal Payment Switch (POX over TCP/IP)

---

c. Architecture Diagram (Description)



| Interbank (ISO8583) |

| Credit Card (SOAP) |

| Payment Switch (POX) |

|

| RDBMS / Storage |

---

d. Architectural Patterns Used

1. **3-Tier Architecture**
   Separates client, business logic, and data access layers for maintainability and scalability.
2. **Model-View-Controller (MVC)**
   Used in front-end apps to separate the UI (View), data (Model), and logic (Controller).
3. **Service-Oriented Architecture (SOA)**
   Used to integrate with heterogeneous back-end systems via services (REST, SOAP, ISO 8583).

---

2. Browser-Based Internet Banking Architecture

Recommended Pattern: Model-View-Controller (MVC)

- **Reason:**
  - **Client-side MVC** (e.g., Angular, React):
    - View: HTML/CSS UI
    - Controller: JavaScript code (input validation, routing)
    - Model: JSON-based data representation from REST APIs

- Allows **faster user interaction**, **better responsiveness**, and **offline validation** before server calls.

---

3. Integration with External Systems

Recommended Architectural Pattern: Service-Oriented Architecture (SOA)

- SOA supports **distributed services** with **loose coupling** and **interoperability**.

Integration Patterns

1. **Adapter Pattern** (Structural)
    - Adapts various protocols like SOAP, REST, ISO8583 into a common internal service format.
2. **Facade Pattern** (Structural)
    - Hides complexity and differences between service interfaces; exposes unified operations like TransferFunds().
3. **Service Locator Pattern**
    - Allows dynamic lookup and connection to services like credit system, payment gateways.

---

4. Notifications for Mobile Banking

Nature of Communication: Asynchronous

- **Push Notifications:** Firebase/APNs – asynchronously triggered
- **SMS Alerts:** Triggered via messaging gateway

Recommended Pattern: Publish and Subscribe

- **Why:**
    - Asynchronous by nature
    - Decouples notification producers (e.g., transaction module) and consumers (e.g., SMS, mobile push)
    - High scalability

Example:

- Publisher: Transaction Engine
- Subscribers: Mobile Notification Service, SMS Gateway

---

5. Managing Transaction Concurrency & ACID

Recommended Tier: Business Logic Tier

- **Why:**
    - Central place to coordinate **transaction boundaries**
    - Implements concurrency control (locks, isolation levels)
    - Ensures **ACID** before committing to the database

Techniques:

- Pessimistic / Optimistic Locking
- Transaction Manager (e.g., JTA in Java EE)
- Two-Phase Commit (for multi-system consistency)

Assumptions

- All external systems are reachable and expose the required interfaces.
- The workflow engine handles approval-based operations (e.g., loan requests).
- APIs are stateless and protected using secure authentication (OAuth2 + 2FA).

Let me know if you'd like a PowerPoint version or a diagram generated visually!

✅ Proposed Architecture for SriBank's Internet and Mobile Banking Platform

The most suitable architecture for SriBank's scenario is a **Multi-tiered, Service-Oriented Architecture (SOA)** integrated with **3-Tier and MVC patterns**, ensuring **scalability, integration, modularity, and security**.

🔧 Architecture Type:

- **Main Architecture:**
  Service-Oriented Architecture (SOA) – to integrate multiple back-end services with different protocols (REST, SOAP, ISO 8583, POX).
- **Supporting Patterns:**
  - **3-Tier Architecture** – Presentation, Business Logic, Data/Service Integration.
  - **Model-View-Controller (MVC)** – for client-side responsiveness and separation of concerns.
  - **Publish-Subscribe** – for asynchronous mobile and SMS notifications.

🧱 Architecture Layers and Tiers

1. Presentation Tier (Client Layer)

- **Mobile App (iOS/Android)**
- **Web App (Browser-based, responsive UI)**
- **MVC structure** on the client side to handle:
  - View: UI rendering
  - Controller: Input validation, user events
  - Model: Data from APIs

2. Business Logic Tier (Application Layer)

- **RESTful API Server**
- **Authentication and 2FA Service**
- **Transaction Processor**

- **Workflow Engine** (for non-real-time processes like loan or service requests)
- **Notification Dispatcher** (Publish-Subscribe for push/SMS)

3. Integration & Data Tier (Back-End)

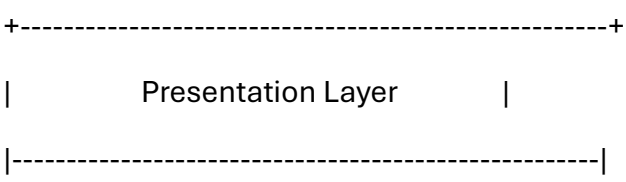
- **Relational Database** (for internal data)
- **External Systems Integration:**
    - **Core Banking System** – via **REST**
    - **Inter-Bank Payment Gateway** – via **ISO 8583**
    - **Credit Card System** – via **SOAP**
    - **Internal Payment Switch** – via **POX over TCP/IP**

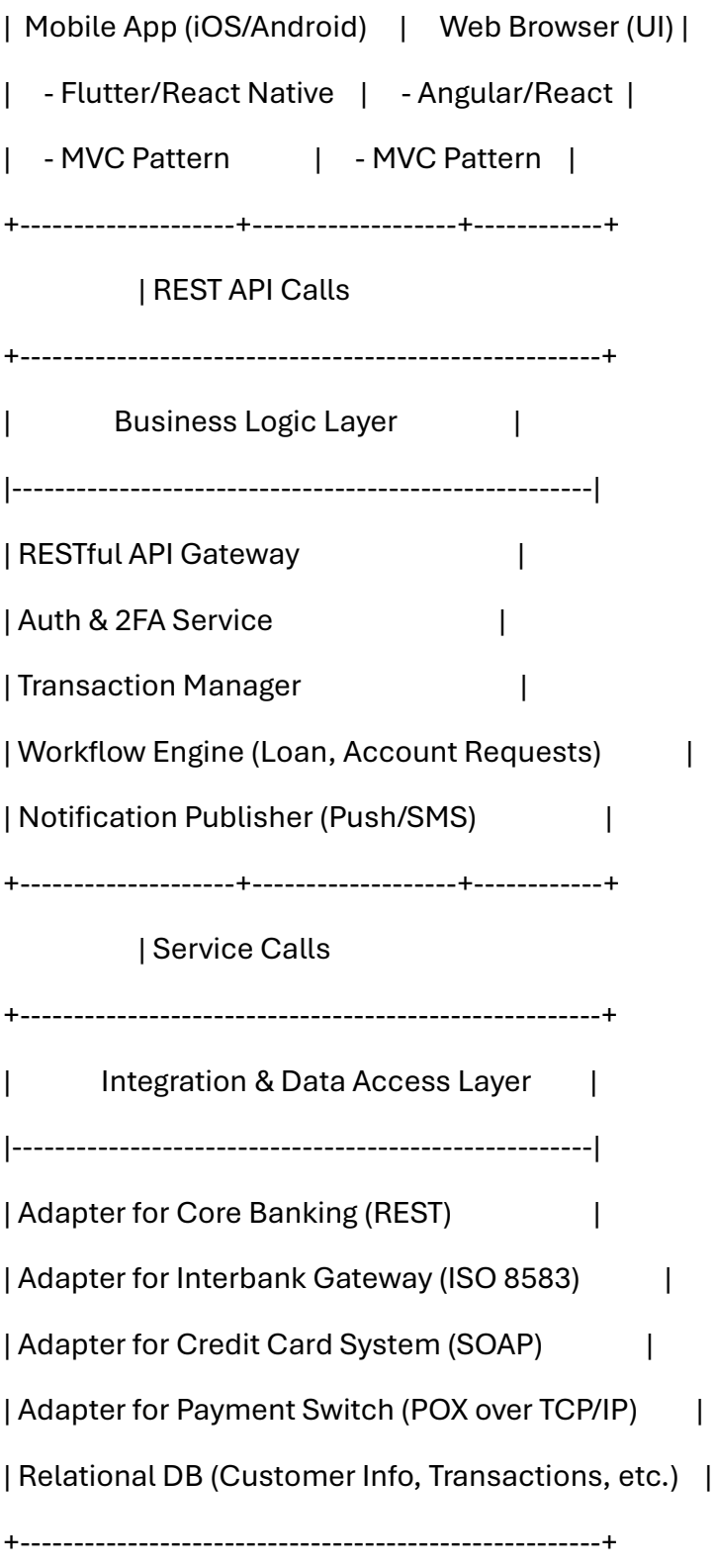
📦 Key Components and Technologies

Tier	Components	Technologies Used
Presentation (Client)	Web, Mobile Apps, Controllers	React, Flutter, Angular, HTML5
Business Logic	API Gateway, Auth Service, Transaction Service	Spring Boot, .NET Core, JWT/OAuth2
	Workflow Engine, Notification System	Camunda, Firebase Cloud Messaging, SMS
Integration Layer	Adapters (REST, SOAP, ISO8583, POX)	Apache Camel, WSDL, TCP/IP parsers
Data Tier	Customer/Account DB, External Service Data	PostgreSQL/MySQL, External APIs

---

🗂️ Architecture Diagram (Text View)





✅ **Why This Architecture?**

- **Scalability:** Can scale each layer independently.
- **Interoperability:** Supports integration with various external systems using SOA principles.
- **Security:** Centralized authentication and 2FA.
- **Responsiveness:** Client MVC and asynchronous Pub/Sub improve UX.
- **Separation of Concerns:** Clear division of responsibilities across tiers.

Let me know if you'd like this visualized as a diagram or exported to a presentation/document format.